BLG411E SOFTWARE ENGINEERING / THE GOYS

İTÜ ShareMaxxing Database Plan

PostgreSQL + Prisma | Proposed design v1.2 | 30 October 2026

A shared relational database for verified student accounts, intercity rides, anonymous seat funding, housing and the student forum. This document gives the tables your team needs, complete SQL definitions, relationships and the transaction rules that the backend must enforce.

Recommended starting point

Use **18 feature tables and one administrative audit table** in one PostgreSQL database. Keep the account role in the user row (USER or ADMIN); driver, passenger and donor are actions a student performs, not separate account types.

Area	Tables	Plan coverage
Accounts and geography	5	T-01 to T-03; driver payee key; location reference data
Rides and anonymous funding	3	T-05 to T-09; T-12 and T-13
Payments and ledger	3	T-10, T-11 and T-14
Reputation	2	T-15 and T-16
Housing and forum	3	T-18, T-19 and T-21
Moderation, notifications, audit	3	T-17 and T-22 to T-26

How to use this document

Pages 2-4: table map, relationships and design decisions.
Pages 5-15: complete core SQL, indexes and views.
Pages 16-19: backend transactions: booking, payment, housing, forum and moderation.
Pages 20-21: Prisma handoff, validation and references.

Status: an implementation proposal derived from the supplied project plan. Assumptions are labeled on page 4. SQL constraints cover row-level integrity; the service transactions on pages 16-19 are also required. This is a database plan, not a deployed database.

What changed in v1.2

- Optional extensions removed: waitlist, forum votes, course tags and vector search (their tables and SQL are gone).
- New backend contracts for housing, forum, reports, AI flags and admin actions (pages 18-19).
- Dropped the three extra delete-speed indexes to keep the migration small.
- Everything from v1.1 stays: UUID keys, USER/ADMIN roles, driver_submerchants, no currency columns, no seat_alias, ACTIVE / REVIEWING / SOLVED cases.

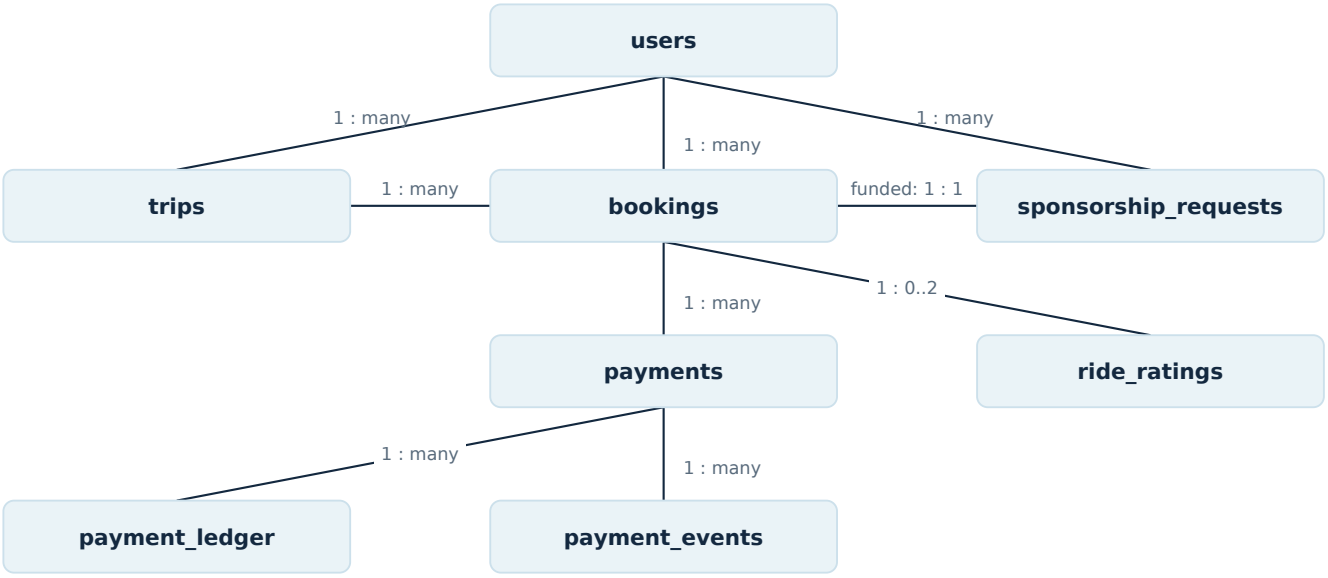
Table inventory

Table	What one row represents	Requirements
users	One user account and profile; USER or ADMIN role.	T-01 to T-03
email_verification_codes	One pending sign-up code per user; replaced on resend, deleted on success.	T-01
driver_submerchants	One driver’s private iyzico sandbox payee key.	T-10, T-11
cities	One Turkish province/city, using its plate code.	T-05, T-06
istanbul_districts	One İstanbul district used by housing listings.	T-18
trips	One driver offer with two cities, time, capacity and price.	T-05, T-06, T-09
bookings	One passenger reserving one seat on one trip.	T-07, T-09, T-15
sponsorship_requests	One private seat-funding request and its pool state.	T-12, T-13
payments	One provider payment attempt; payer can be a donor.	T-10, T-11, T-13
payment_events	One authenticated provider event/response for deduplication.	T-10, T-11, T-14
payment_ledger	One immutable successful PAYMENT/HOLD/RELEASE/REFUND event.	T-14
ride_ratings	One direction of a driver/passenger rating pair.	T-16
no_show_reports	One claim that a booking participant did not attend.	T-15
housing_listings	One room offer or room-needed advert.	T-18
forum_categories	One seeded category label and code.	T-19
forum_posts	One question or one answer linked to a question.	T-19, T-21
moderation_cases	One user report or AI flag with a concrete target and a review status.	T-17, T-24, T-25
notifications	One in-app event for one recipient.	T-22
admin_audit_logs	One admin role change, ban or reactivation; supporting design addition.	T-26

No separate tables are needed for ride history, ratings averages, free-seat totals, dashboards or donor balances. Derive them from bookings, ratings, payments and the ledger. JWT access tokens alone do not require a session table. Moderation decisions need no action table: a case records its own outcome.

Relationships and ownership

Every UUID foreign key in the schemas points to the owning row. The diagram highlights the ride and payment relationships; the remaining relationships are listed underneath.



Parent	Child / cardinality	Meaning
users	trips, bookings, payments: 1 to many	A user can drive, ride and pay on different trips.
users	driver_submerchants: 1 to 0..1	Only drivers who offer paid trips need a payee key.
users	email_verification_codes: 1 to 0..1	At most one pending code per user.
sponsorship_requests	bookings: 1 to 0..1	An open request has no seat; claiming funding creates its booking.
forum_posts	forum_posts: 1 question to many answers	Answers reference a question through parent_post_id; deleting the question deletes them.
forum_categories	forum_posts: 1 to many questions	Answers inherit their question’s category.
istanbul_districts	housing_listings: 1 to many	Housing is restricted to seeded İstanbul districts.
users, trips, housing_listings, forum_posts	moderation_cases: 1 to many	An open case targets exactly one of them; reviewed_by names the admin who took the case.
users	notifications: 1 to many	Only the notification’s recipient may read it.

Key terms: PK = unique row identifier; FK = reference to another table; UNIQUE = duplicate prevention; CHECK = rule PostgreSQL validates. A column without NOT NULL is optional. A join table connects many rows on both sides.

Design decisions and boundaries

Decision	Proposed behavior
One booking = one seat	The plan speaks of booking a seat. Multiple-seat group purchases are excluded from this first schema.
Completion is per driver/passenger pair	Driver confirms the trip once; each passenger confirms their own booking. Release that booking’s payment after both confirmations, rather than waiting for every passenger.
Seat reservations expire	PENDING_PAYMENT bookings hold a seat for 15 minutes (configurable). A background worker expires unpaid reservations. Creating an OPEN funding request holds no seat.
Cancellation and refunds	Allow ordinary cancellation before departure and before release; refund the original payer, including a donor. Later disputes are reviewed manually. Automatic post-release clawback is outside this design.
No-show is an allegation	Create a report and a dispute; do not immediately punish a user or release money. An admin confirms or rejects the claim.
Funding-request privacy	No public free-text plea, name, route or date. Pool exposes only request UUID, amount (TL) and OPEN status. Amounts can still allow inference; anonymity is enforced as an API access policy.
Funded-seat privacy	Drivers never see a funded passenger: no name, alias, contact or profile link. A funded booking counts toward occupied seats but is left out of the driver’s passenger list. The driver therefore cannot rate that passenger or file a no-show claim for it; the funded passenger can still rate the driver and report a driver no-show.
Quote and trip edits	Freeze route, departure and seat price after the first live request/booking; allow contact/description edits. Capacity can change only if it stays above occupied seats.
Roles and bans	Only two roles exist: USER and ADMIN. An admin either deletes reported content or bans the user. A ban sets status DEACTIVATED and is written to admin_audit_logs.
Deletion and history	Keep financial and ride evidence. Banned users stay as DEACTIVATED rows. Forum posts and housing listings are hard-deleted, and a deleted question takes its answers with it. Never cascade deletion of payments, bookings or audit data.

Types and conventions

All surrogate primary keys are UUIDs generated with **gen_random_uuid()**; no table uses auto-increment or identity columns. Natural keys stay natural: cities use plate codes (İstanbul = 34), ride_ratings uses (booking_id, direction), and email_verification_codes and driver_submerchants use user_id. Money is integer **kuruş** in TL: 25000 = 250.00 TL; never floating point, and no table has a currency column. Timestamps use **timestamptz**; display them in Europe/İstanbul. All SQL below is PostgreSQL 14+ syntax; pin the deployed PostgreSQL and Prisma versions.

Unspecified FK deletion behavior is PostgreSQL NO ACTION: hard deletion of referenced records is blocked. The only exceptions are ON DELETE CASCADE from a question to its answers and ON DELETE SET NULL for notification and moderation-case references to a deleted post or listing. Mutable tables use updated_at, maintained by Prisma @updatedAt or explicitly by raw SQL. Row checks cannot enforce rules involving other rows; those rules are assigned to service transactions. [S2, S3]

Accounts and location schemas

Only verified ACTIVE users may access application features; a banned user is DEACTIVATED. Email is canonical lowercase. A user’s profile stays in the same row; do not return email/password fields from public profile endpoints.

```
CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email text NOT NULL UNIQUE,
  password_hash text NOT NULL,
  display_name varchar(100) NOT NULL,
  bio varchar(500),
  role text NOT NULL DEFAULT 'USER'
    CHECK (role IN ('USER','ADMIN')),
  status text NOT NULL DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE','DEACTIVATED')),
  email_verified_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (email = lower(email)),
  CHECK (email ~ '^[a-z0-9._%+-]+@itu[.]edu[.]tr$'),
  CHECK (length(trim(display_name)) > 0)
);

CREATE TABLE driver_submerchants (
  user_id uuid PRIMARY KEY REFERENCES users(id),
  iyzico_submerchant_key text NOT NULL UNIQUE,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE email_verification_codes (
    user_id UUID PRIMARY KEY REFERENCES users(id),
    code_hash TEXT NOT NULL,
    failed_attempts SMALLINT NOT NULL DEFAULT 0
        CHECK (failed_attempts BETWEEN 0 AND 5),
    expires_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE cities (
  code smallint PRIMARY KEY CHECK (code BETWEEN 1 AND 81),
  name varchar(80) NOT NULL UNIQUE
);
CREATE TABLE istanbul_districts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name varchar(80) NOT NULL UNIQUE
);
```

Service rules: hash passwords (for example, Argon2id) and HMAC/hash short verification codes with a server-held secret. Sign-up and resend upsert the user’s single code row: new code_hash, failed_attempts = 0 and expires_at = now() + 10 minutes. Lock the row (SELECT ... FOR UPDATE) while checking it. Reject without comparing once failed_attempts is 5; otherwise each wrong code adds 1. On success delete the row and set users.email_verified_at. Delete expired rows with a periodic job and rate-limit by email/IP. Email changes and password resets are not specified. driver_submerchants.iyzico_submerchant_key maps a driver to a sandbox payee; configure test fixtures before allowing paid trips. No real identity or banking data is needed in this course database. [S7]

Trip and booking schemas

```
CREATE TABLE trips (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  driver_id uuid NOT NULL REFERENCES users(id),
  origin_city_code smallint NOT NULL REFERENCES cities(code),
  destination_city_code smallint NOT NULL REFERENCES cities(code),
  departure_at timestamptz NOT NULL,
  seat_capacity smallint NOT NULL CHECK (seat_capacity > 0),
  price_minor bigint NOT NULL CHECK (price_minor > 0),
  description text,
  contact_details varchar(250),
  status text NOT NULL DEFAULT 'PUBLISHED'
    CHECK (status IN ('PUBLISHED','COMPLETED','CANCELLED')),
  driver_completed_at timestamptz,
  cancelled_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (origin_city_code <> destination_city_code),
  CHECK (origin_city_code = 34 OR destination_city_code = 34)
);

-- The sponsorship FK is added after its table on page 7.
CREATE TABLE bookings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  trip_id uuid NOT NULL REFERENCES trips(id),
  passenger_id uuid NOT NULL REFERENCES users(id),
  sponsorship_request_id uuid UNIQUE,
  price_minor bigint NOT NULL CHECK (price_minor > 0),
  status text NOT NULL DEFAULT 'PENDING_PAYMENT'
    CHECK (status IN ('PENDING_PAYMENT','CONFIRMED','COMPLETED',
                      'CANCELLED','EXPIRED','DISPUTED')),
  reservation_expires_at timestamptz NOT NULL,
  passenger_completed_at timestamptz,
  completed_at timestamptz,
  cancelled_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (reservation_expires_at > created_at),
  CHECK (status <> 'COMPLETED' OR
    (completed_at IS NOT NULL AND passenger_completed_at IS NOT NULL))
);
CREATE UNIQUE INDEX one_active_booking_per_passenger
  ON bookings(trip_id, passenger_id)
  WHERE status IN ('PENDING_PAYMENT','CONFIRMED','COMPLETED','DISPUTED');
```

Capacity: seat_capacity counts passenger seats offered, excluding the driver. Count PENDING_PAYMENT, CONFIRMED, COMPLETED and DISPUTED bookings as occupied, funded ones included. Expire stale reservations under a trip-row lock; do not make the index depend on now(). No cached free_seats column is stored. The same lock protects inserts, cancellations and capacity edits.

Service rules: driver ≠ passenger; both are verified/active; departure is in the future when booking; price is copied from the accepted quote and is always TL in kuruş. A driver confirmation alone does not complete every booking. Ordinary cancellations preserve the row and release its seat. A booking with sponsorship_request_id set is a funded seat and is never shown to the driver.

Funding requests and payment attempts

```
CREATE TABLE sponsorship_requests (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  trip_id uuid NOT NULL REFERENCES trips(id),
  recipient_id uuid NOT NULL REFERENCES users(id),
  amount_minor bigint NOT NULL CHECK (amount_minor > 0),
  status text NOT NULL DEFAULT 'OPEN'
    CHECK (status IN ('OPEN','FUNDING','FUNDED','CANCELLED','EXPIRED')),
  expires_at timestamptz NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (expires_at > created_at)
);
CREATE UNIQUE INDEX one_live_funding_request
  ON sponsorship_requests(trip_id, recipient_id)
  WHERE status IN ('OPEN','FUNDING','FUNDED');
ALTER TABLE bookings ADD CONSTRAINT booking_sponsorship_fk
  FOREIGN KEY (sponsorship_request_id) REFERENCES sponsorship_requests(id);

CREATE TABLE payments (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id uuid NOT NULL REFERENCES bookings(id),
  payer_id uuid NOT NULL REFERENCES users(id),
  amount_minor bigint NOT NULL CHECK (amount_minor > 0),
  provider text NOT NULL DEFAULT 'IYZICO_SANDBOX'
    CHECK (provider = 'IYZICO_SANDBOX'),
  provider_payment_id text UNIQUE,
  provider_transaction_id text UNIQUE,
  provider_refund_id text,
  idempotency_key uuid NOT NULL UNIQUE,
  status text NOT NULL DEFAULT 'CREATED'
    CHECK (status IN ('CREATED','PENDING','HELD','RELEASE_PENDING',
      'RELEASED','REFUND_PENDING','REFUNDED','FAILED')),
  failure_code text,
  next_retry_at timestamptz,
  retry_count integer NOT NULL DEFAULT 0 CHECK (retry_count >= 0),
  held_at timestamptz,
  released_at timestamptz,
  refunded_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX one_nonfailed_payment_per_booking
  ON payments(booking_id) WHERE status <> 'FAILED';
```

Private links: a funded booking references its request; its payment references the donor as payer. For an ordinary booking, payer = passenger. Only a definitively failed attempt allows a new attempt row. An uncertain provider timeout stays PENDING and is reconciled before retrying. Refunded bookings are not reused; rebooking creates a new booking. Every amount is TL; the iyzico adapter sends the constant currency TRY to the provider, so no currency is stored.

Provider events and payment ledger

```
CREATE TABLE payment_events (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id uuid NOT NULL REFERENCES payments(id),
  dedupe_key text NOT NULL UNIQUE,
  event_type text NOT NULL,
  payload jsonb NOT NULL DEFAULT '{}'::jsonb,
  received_at timestamptz NOT NULL DEFAULT now(),
  processed_at timestamptz,
  processing_error text
);

CREATE TABLE payment_ledger (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id uuid NOT NULL REFERENCES payments(id),
  event_type text NOT NULL
    CHECK (event_type IN ('PAYMENT','HOLD','RELEASE','REFUND')),
  amount_minor bigint NOT NULL CHECK (amount_minor > 0),
  source_event_id uuid REFERENCES payment_events(id),
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (payment_id, event_type)
);
-- This MVP allows a full release OR a full refund, not both.
CREATE UNIQUE INDEX one_terminal_ledger_event
  ON payment_ledger(payment_id)
  WHERE event_type IN ('RELEASE','REFUND');
```

What the ledger means

Event	When inserted	Financial interpretation
PAYMENT	Provider confirms a successful collection.	Gross cash collected from the payer.
HOLD	In the same DB transaction as PAYMENT.	The same amount is allocated to a held booking.
RELEASE	Provider confirms release/approval after completion.	That held amount becomes payable to the driver.
REFUND	Provider confirms the full refund to the original payer.	That held amount is returned to the payer.

All amounts are positive event magnitudes. **Never sum all four event types as revenue.** Gross received = ΣPAYMENT; held = ΣHOLD - ΣRELEASE - ΣREFUND; released = ΣRELEASE; refunded = ΣREFUND. For each paid booking, gross received must equal held + released + refunded. This is a lifecycle/reconciliation ledger, not a double-entry accounting system.

Service rules enforce matching payment/ledger amounts, valid event order and provider identifiers. Ledger rows are inserted only after confirmed provider results. Duplicate callbacks hit unique keys and must not charge, notify or insert ledger rows twice. Sanitize payloads; never store card numbers, CVV, API secrets or a raw checkout request.

> iyzico distinguishes a payment ID from a payment item/split transaction ID. Keep both; item approval uses paymentTransactionId. HELD/RELEASED are application states mapped through the chosen sandbox integration, not a claim that a SQL row itself holds money. [S5]

Ratings and no-show schemas

```
CREATE TABLE ride_ratings (
  booking_id uuid NOT NULL REFERENCES bookings(id),
  direction text NOT NULL CHECK (direction IN
    ('DRIVER_TO_PASSENGER','PASSENGER_TO_DRIVER')),
  score smallint NOT NULL CHECK (score BETWEEN 1 AND 5),
  review varchar(1000),
  created_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (booking_id, direction)
);

CREATE TABLE no_show_reports (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  booking_id uuid NOT NULL REFERENCES bookings(id),
  missing_party text NOT NULL
    CHECK (missing_party IN ('DRIVER','PASSENGER')),
  details varchar(1000),
  status text NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING','CONFIRMED','REJECTED')),
  reviewed_by uuid REFERENCES users(id),
  reviewed_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (booking_id, missing_party),
  CHECK ((status = 'PENDING' AND reviewed_at IS NULL
          AND reviewed_by IS NULL)
      OR (status <> 'PENDING' AND reviewed_at IS NOT NULL
          AND reviewed_by IS NOT NULL))
);
```

Do not duplicate participant identity

For DRIVER_TO_PASSENGER, reviewer = trips.driver_id and subject = bookings.passenger_id. Reverse those for PASSENGER_TO_DRIVER. Derive these IDs through joins instead of storing freely editable reviewer/subject IDs. For a no-show report, the reporter is the other participant; for example, missing_party = DRIVER is reported by the passenger. reviewed_by must be an ADMIN.

Submission and visibility

Only the appropriate participant can create each direction, and only for a COMPLETED booking. Make submitted ratings immutable. The primary key permits at most one rating in each direction. The view on page 15 exposes a booking’s rating rows only when both exist, except for funded bookings, where the driver cannot rate and the passenger’s rating is exposed at once. Calculate public reputation from that revealed set; do not leak the first rating through counts, averages or notification payloads.

No-show and disputes

A participant may file a no-show claim after scheduled departure while settlement is still held. In one transaction, lock the trip, mark the affected booking DISPUTED, insert the report and queue a notification. Admins decide whether to restore confirmation, cancel/refund or complete/release. That policy is a human decision; a no-show report alone is not proof.

For funded bookings the driver is never shown the passenger, so the service rejects a DRIVER_TO_PASSENGER rating and a PASSENGER no-show claim on them. The funded passenger can still rate the driver and report a driver no-show. Public ride history should use aggregate counts without route/date detail that could identify a funded recipient. An admin may resolve the internal participant for safety review.

Housing and forum categories

```
CREATE TABLE housing_listings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id uuid NOT NULL REFERENCES users(id),
  district_id uuid NOT NULL REFERENCES istanbul_districts(id),
  listing_type text NOT NULL
    CHECK (listing_type IN ('HAVE_ROOM','NEED_ROOM')),
  title varchar(160) NOT NULL CHECK (length(trim(title)) > 0),
  description text NOT NULL CHECK (length(trim(description)) > 0),
  contact_details varchar(250),
  status text NOT NULL DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE','CLOSED')),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE forum_categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code varchar(40) NOT NULL UNIQUE,
  label_tr varchar(80) NOT NULL
);
INSERT INTO forum_categories(code, label_tr) VALUES
  ('COURSE','Dersler'), ('GENERAL','Genel'),
  ('CAFETERIA','Yemekhane'), ('HOUSING','Barınma'),
  ('TRANSPORT','Ulaşım'), ('CAMPUS_LIFE','Kampüs Yaşamı');
```

Housing scope stays small

The source requires listing type, district, content and optional contact details. Rent collection, deposits and rental contracts have no tables because they are out of scope. Add rent/room-size/image fields later only if the requirements are explicitly expanded.

Editing and deleting

Authors can edit, close or delete their own listings; admins can delete any listing. A delete removes the row (hard delete). The step-by-step rules are on page 18 and the AI check is on page 19.

Seed data

Seed all 81 city codes/names and the current İstanbul district list before enabling posting. Seed İstanbul with code 34. Keep lookup data migration-owned; users cannot create arbitrary cities or districts. These lists are implementation data, not sample free-text columns.

Optional contact details are returned only to verified ACTIVE students. Listing contact details are not copied automatically from a private account field. A CLOSED listing remains in the owner’s history but is omitted from ordinary browsing.

Forum posts and notifications

```
CREATE TABLE forum_posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id uuid NOT NULL REFERENCES users(id),
  kind text NOT NULL CHECK (kind IN ('QUESTION','ANSWER')),
  parent_post_id uuid REFERENCES forum_posts(id) ON DELETE CASCADE,
  category_id uuid REFERENCES forum_categories(id),
  title varchar(180),
  body text NOT NULL CHECK (length(trim(body)) > 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (parent_post_id IS DISTINCT FROM id),
  CHECK ((kind = 'QUESTION' AND parent_post_id IS NULL
    AND category_id IS NOT NULL AND title IS NOT NULL
    AND length(trim(title)) > 0)
    OR (kind = 'ANSWER' AND parent_post_id IS NOT NULL
    AND category_id IS NULL AND title IS NULL))
);

CREATE TABLE notifications (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  recipient_id uuid NOT NULL REFERENCES users(id),
  event_key text NOT NULL,
  kind text NOT NULL CHECK (kind IN ('BOOKING','CANCELLATION',
    'RIDE_REMINDER','PAYMENT','FUNDING','FORUM_REPLY','MODERATION')),
  booking_id uuid REFERENCES bookings(id),
  payment_id uuid REFERENCES payments(id),
  forum_post_id uuid REFERENCES forum_posts(id) ON DELETE SET NULL,
  sponsorship_request_id uuid REFERENCES sponsorship_requests(id),
  payload jsonb NOT NULL DEFAULT '{}'::jsonb,
  read_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (recipient_id, event_key)
);
```

Forum service rules: an ANSWER must point to a QUESTION, never another answer. Its category is inherited. Deleting a question deletes its answers (ON DELETE CASCADE). Treat kind/parent/category identity as immutable except an explicit category edit on a question. updated_at changes on every edit. Who may do what is on page 18.

Notifications: insert in the same transaction as the business event. A deterministic event_key such as booking:<UUID>:confirmed prevents duplicate delivery. Payload contains display-safe text, not donor/recipient identities or private routes. Build links from the explicit FK columns; a deleted post leaves forum_post_id NULL.

Moderation queue schema

```
CREATE TABLE moderation_cases (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  source text NOT NULL CHECK (source IN ('USER','AI')),
  reporter_id uuid REFERENCES users(id),
  target_user_id uuid REFERENCES users(id),
  target_trip_id uuid REFERENCES trips(id),
  target_housing_id uuid REFERENCES housing_listings(id) ON DELETE SET NULL,
  target_post_id uuid REFERENCES forum_posts(id) ON DELETE SET NULL,
  reason text NOT NULL CHECK (length(trim(reason)) > 0),
  ai_model text,
  ai_score numeric(5,4) CHECK (ai_score BETWEEN 0 AND 1),
  status text NOT NULL DEFAULT 'ACTIVE'
    CHECK (status IN ('ACTIVE','REVIEWING','SOLVED')),
  reviewed_by uuid REFERENCES users(id),
  resolution text CHECK (resolution IN
    ('DISMISSED','CONTENT_DELETED','USER_BANNED')),
  resolution_note text,
  resolved_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  CHECK (num_nonnulls(target_user_id, target_trip_id,
    target_housing_id, target_post_id) <= 1),
  CHECK (status = 'SOLVED' OR num_nonnulls(target_user_id,
    target_trip_id, target_housing_id, target_post_id) = 1),
  CHECK ((source = 'USER' AND reporter_id IS NOT NULL)
      OR (source = 'AI' AND reporter_id IS NULL AND ai_model IS NOT NULL
        AND (status = 'SOLVED'
          OR num_nonnulls(target_post_id, target_housing_id) = 1))),
  CHECK ((status = 'ACTIVE' AND reviewed_by IS NULL
          AND resolution IS NULL AND resolved_at IS NULL)
      OR (status = 'REVIEWING' AND reviewed_by IS NOT NULL
          AND resolution IS NULL AND resolved_at IS NULL)
      OR (status = 'SOLVED' AND resolution IS NOT NULL
          AND resolved_at IS NOT NULL
          AND (resolution = 'CONTENT_DELETED' OR reviewed_by IS NOT NULL)))
);
```

Case lifecycle

A case starts ACTIVE in the queue. An admin who opens it sets REVIEWING and reviewed_by. The admin then deletes the reported post or listing, bans the user or dismisses the report (steps on page 19), and the case becomes SOLVED with resolution CONTENT_DELETED, USER_BANNED or DISMISSED, an optional resolution_note and resolved_at.

Why explicit target columns?

A generic target_type + target_id pair cannot have a normal FK to several different tables. Four nullable target FKs plus the CHECKs make an open case point at exactly one existing object. Deleting a post or listing sets its target column to NULL, so SOLVED cases stay as history. The CHECK blocks that while a case is open: in the same transaction, first mark every open case on the content (and on a deleted question’s answers) SOLVED with CONTENT_DELETED (statement on page 18), then delete. reviewed_by may stay empty only on that author-deleted path.

Human authority stays in the service

AI creates a case and suggests a reason/score; it never deletes content or bans users itself. How the AI job avoids stale flags is on page 19.

AI scores are optional hints. Their scale and meaning depend on the selected provider; do not present them as a proven probability of misconduct. The score column permits 0..1 only when the adapter normalizes to that range.

Admin audit log and append-only tables

```
CREATE TABLE admin_audit_logs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_id uuid NOT NULL REFERENCES users(id),
  target_user_id uuid NOT NULL REFERENCES users(id),
  action text NOT NULL CHECK (action IN
    ('CHANGE_ROLE','BAN','REACTIVATE')),
  before_state jsonb NOT NULL,
  after_state jsonb NOT NULL,
  reason text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE FUNCTION reject_record_mutation() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'append-only table: %', TG_TABLE_NAME;
  RETURN NULL;
END;
$$;
CREATE TRIGGER ledger_is_append_only
  BEFORE UPDATE OR DELETE ON payment_ledger
  FOR EACH ROW EXECUTE FUNCTION reject_record_mutation();
CREATE TRIGGER admin_audit_is_append_only
  BEFORE UPDATE OR DELETE ON admin_audit_logs
  FOR EACH ROW EXECUTE FUNCTION reject_record_mutation();
CREATE TRIGGER ratings_are_immutable
  BEFORE UPDATE OR DELETE ON ride_ratings
  FOR EACH ROW EXECUTE FUNCTION reject_record_mutation();
```

Service rules: the BAN, REACTIVATE and CHANGE_ROLE transactions are on page 19. Allow only approved safe fields in audit JSON (for example role and status); never hash/password/token/payment payload snapshots.

Where each decision is recorded

Event	Recorded in
Role change	admin_audit_logs (CHANGE_ROLE)
Ban or reactivation	admin_audit_logs (BAN / REACTIVATE) and users.status
Reported post or listing deleted	moderation_cases.resolution = CONTENT_DELETED; the row itself is gone
Report dismissed	moderation_cases.resolution = DISMISSED
Ban caused by a report	both: resolution = USER_BANNED and a BAN audit row

Access boundaries

Any user can create a report and see a safe status for their own reports. Only ADMIN accounts review cases, delete reported content, ban or reactivate users, change roles and use the payment ledger and askıda bilet administration. There is no moderator role; check users.role on every admin endpoint.

The audit table is a recommended supporting addition, not a separate feature in the source plan. Runtime DB credentials must not own tables or have DDL/TRUNCATE privileges; triggers do not prevent an owner from changing the schema. Migrations use separate credentials.

Indexes for the main screens

```
CREATE INDEX trips_search_idx ON trips
  (origin_city_code, destination_city_code, departure_at)
  WHERE status = 'PUBLISHED';
CREATE INDEX trips_driver_idx ON trips(driver_id, departure_at);
CREATE INDEX bookings_trip_status_idx ON bookings(trip_id, status);
CREATE INDEX bookings_passenger_idx ON bookings(passenger_id, created_at DESC);
CREATE INDEX booking_expiry_idx ON bookings(reservation_expires_at)
  WHERE status = 'PENDING_PAYMENT';
CREATE INDEX funding_pool_idx ON sponsorship_requests(created_at, id)
  WHERE status = 'OPEN';
CREATE INDEX funding_recipient_idx ON sponsorship_requests(recipient_id);
CREATE INDEX payments_payer_idx ON payments(payer_id, created_at DESC);
CREATE INDEX payment_retry_idx ON payments(next_retry_at)
  WHERE status IN ('PENDING','RELEASE_PENDING','REFUND_PENDING');
CREATE INDEX unprocessed_payment_event_idx ON payment_events(received_at)
  WHERE processed_at IS NULL;
CREATE INDEX housing_browse_idx ON housing_listings
  (district_id, listing_type, created_at DESC) WHERE status = 'ACTIVE';
CREATE INDEX housing_author_idx ON housing_listings(author_id);
CREATE INDEX forum_browse_idx ON forum_posts(category_id, created_at DESC)
  WHERE kind = 'QUESTION';
CREATE INDEX forum_answers_idx ON forum_posts(parent_post_id, created_at);
CREATE INDEX forum_author_idx ON forum_posts(author_id);
CREATE INDEX forum_keyword_idx ON forum_posts USING gin (
  to_tsvector('simple', coalesce(title, '') || ' ' || body));
CREATE INDEX moderation_queue_idx ON moderation_cases(status, created_at);
CREATE INDEX unread_notifications_idx ON notifications(recipient_id, created_at DESC)
  WHERE read_at IS NULL;
```

Keyword search

```
-- Bind $1 as user input. Apply category/pagination as needed.
SELECT id, parent_post_id, kind, title
FROM forum_posts
WHERE to_tsvector('simple', coalesce(title, '') || ' ' || body)
      @@ plainto_tsquery('simple', $1)
ORDER BY created_at DESC, id LIMIT 30;
```

The built-in simple text-search configuration gives token-based keyword matching with no extra extension. It does not provide Turkish stemming or substring matching. Use the same expression in index and query. If substring search is later required, pg_trgm is a separate extension option. [S6]

Derived views and safe read models

```
CREATE VIEW trip_availability AS
SELECT t.id AS trip_id,
       t.seat_capacity - count(b.id)::integer AS free_seats
FROM trips t LEFT JOIN bookings b ON b.trip_id = t.id
  AND b.status IN ('PENDING_PAYMENT','CONFIRMED','COMPLETED','DISPUTED')
GROUP BY t.id, t.seat_capacity;

CREATE VIEW open_sponsorship_pool AS
SELECT id AS request_id, amount_minor, status
FROM sponsorship_requests
WHERE status = 'OPEN' AND expires_at > now()
  AND EXISTS (SELECT 1 FROM trips t
    WHERE t.id = sponsorship_requests.trip_id
      AND t.status = 'PUBLISHED' AND t.departure_at > now());

CREATE VIEW driver_visible_bookings AS
SELECT b.id AS booking_id, b.trip_id, b.passenger_id, b.status
FROM bookings b
WHERE b.sponsorship_request_id IS NULL;  -- funded seats are never shown

CREATE VIEW revealed_ride_ratings AS
SELECT r.booking_id, r.direction, r.score, r.review, r.created_at
FROM ride_ratings r JOIN bookings b ON b.id = r.booking_id
WHERE b.sponsorship_request_id IS NOT NULL  -- the driver cannot rate these
   OR (SELECT count(*) FROM ride_ratings r2
       WHERE r2.booking_id = r.booking_id) = 2;

CREATE VIEW payment_reconciliation AS
SELECT p.id AS payment_id, p.amount_minor,
  coalesce(sum(l.amount_minor) FILTER
    (WHERE l.event_type = 'PAYMENT'), 0) AS received_minor,
  coalesce(sum(l.amount_minor) FILTER
    (WHERE l.event_type = 'HOLD'), 0)
  - coalesce(sum(l.amount_minor) FILTER
    (WHERE l.event_type IN ('RELEASE','REFUND')), 0) AS held_minor,
  coalesce(sum(l.amount_minor) FILTER
    (WHERE l.event_type = 'RELEASE'), 0) AS released_minor,
  coalesce(sum(l.amount_minor) FILTER
    (WHERE l.event_type = 'REFUND'), 0) AS refunded_minor
FROM payments p LEFT JOIN payment_ledger l ON l.payment_id = p.id
GROUP BY p.id, p.amount_minor;
```

Trip search order

```
SELECT t.id, t.departure_at, t.price_minor, a.free_seats
FROM trips t JOIN trip_availability a ON a.trip_id = t.id
WHERE t.origin_city_code = $1 AND t.destination_city_code = $2
  AND t.status = 'PUBLISHED' AND t.departure_at > now()
ORDER BY (a.free_seats > 0) DESC, t.departure_at, t.id;
```

Views are internal read helpers, not authorization by themselves. Students must not have direct DB access. Use explicit API field allowlists: never serialize a Prisma relation graph into a pool, driver manifest, donor receipt or public profile response. The driver’s passenger list must come from driver_visible_bookings. A pool claim always rechecks availability; an OPEN row is not a guaranteed seat.

Booking and funding transactions

A. Reserve a normal paid seat

1. Require a verified ACTIVE passenger. Start a short Prisma transaction and lock the trip row with SELECT ... FOR UPDATE.
2. Require PUBLISHED, departure in the future, driver ≠ passenger and no live request/booking for the same passenger/trip. Expire eligible stale holds under the same lock.
3. Count occupied bookings. If count ≥ seat_capacity, return “full”; otherwise copy the trip price and insert a PENDING_PAYMENT booking with a 15-minute expiry.
4. Create its CREATED payment row with a server-controlled idempotency key. Insert the pending-booking notification and commit.
5. Call the sandbox provider after commit. Update the attempt to PENDING/HELD only through the verified result handler.

Every path that changes a trip’s occupancy uses the same lock order: **trip → sponsorship request (if any) → booking → payment**. With this protocol, simultaneous requests for the last seat serialize. A read/count followed by an unlocked insert is unsafe. PostgreSQL row locks are the mechanism; the service must consistently use them. [S3]

B. Create and anonymously fund a request

1. The recipient chooses a trip privately. Lock the trip and ensure no active self-booking or live funding request exists. Create an OPEN request with the current seat price; its expiry must be before departure. Do not reserve a seat yet.
2. The pool returns only the safe view from page 15. A donor chooses an opaque request ID and receives no route, date, recipient ID or contact details.
3. Funding locks trip then request. Recheck OPEN, expiry, recipient eligibility, quote and free seats. Donor must be verified/active and different from recipient and driver.
4. In one transaction, set FUNDING, create the recipient’s PENDING_PAYMENT booking with that request FK, and create the donor’s payment attempt. Commit before calling the provider.
5. A successful payment confirms the booking and sets FUNDED. The driver sees nothing: the booking is left out of the passenger list and only reduces the free-seat count. The recipient receives a generic “seat funded” notification.

C. Failure, expiry and competing donors

Only one donor can change OPEN to FUNDING under the row lock. While the hold is valid, failed attempts may be retried by that same donor; other donors cannot take over an active payment attempt. On final failure/expiry, mark the booking EXPIRED and request EXPIRED and release capacity. The recipient can create a new request. Do not reset the same request to OPEN after it already has a booking.

A provider success arriving after expiry/cancellation must never confirm an unreserved seat. Record PAYMENT and HOLD, set REFUND_PENDING and issue a full refund to the payer. Keep the booking inactive. A funding request that is cancelled after being funded moves to CANCELLED; preserve its ledger.

Payment and completion transactions

Entity	Normal state path	Alternative path
Booking	PENDING_PAYMENT → CONFIRMED → COMPLETED	PENDING_PAYMENT → EXPIRED; active → CANCELLED / DISPUTED
Funding request	OPEN → FUNDING → FUNDED	OPEN/FUNDING → EXPIRED; live/funded → CANCELLED
Payment	CREATED → PENDING → HELD → RELEASE_PENDING → RELEASED	Before collection → FAILED; after collection HELD → REFUND_PENDING → REFUNDED

Provider result processing

Authenticate/verify the provider callback or retrieve the provider result server-side; never trust a browser “success” URL. Derive a stable dedupe_key from the provider event identifier or verified operation/payment/result tuple. Resolve it to an existing server-created attempt; validate amount and reference IDs. Insert payment_events once, then lock in the common order and process atomically. A duplicate or out-of-order result is a no-op or reconciliation task, never a blind state overwrite.

On success: add PAYMENT + HOLD together, move payment to HELD and booking to CONFIRMED if the seat is still valid, and mark the event processed. On cancellation: set booking CANCELLED; set a collected payment REFUND_PENDING atomically, but keep an uncollected attempt PENDING until its outcome is known. Call the provider after commit. Only a confirmed refund adds REFUND and sets REFUNDED. A late success on a cancelled/expired booking follows the refund path on page 16.

Completion and release

After scheduled departure, driver sets trips.driver_completed_at. A passenger sets bookings.passenger_completed_at for their own CONFIRMED booking. When both are present and there is no open dispute, set that booking COMPLETED and its HELD payment RELEASE_PENDING in one transaction. Process all newly eligible bookings when the driver confirms. The job calls provider approval, then writes RELEASE and RELEASED only after a verified successful result. Set the trip COMPLETED when all non-cancelled/non-expired bookings are completed and the driver has confirmed. RELEASE means provider-approved gross allocation, not a confirmed bank transfer; fees and bank payouts are outside this sandbox ledger.

Retries and reconciliation

A small scheduled worker scans PENDING, RELEASE_PENDING and REFUND_PENDING rows using next_retry_at/retry_count. Database status is its durable work queue; no additional job table is needed for this scope. On network timeout, retrieve provider state before repeating an operation. The application idempotency key prevents duplicate local commands; do not assume the provider accepts that key. Match payment IDs/item IDs/refund IDs and verify provider state. Compare the reconciliation view with provider results and the admin funding overview.

Privacy and access checks

Passenger: own booking and funding status, no donor fields. Donor: own payment amount/status and a generic funding receipt, no booking/recipient/trip link. Driver: seat/payment state and ordinary passenger details; funded seats are not displayed at all. Admin: case evidence, finance links and management as needed. Never expose provider_transaction_id, internal joins or unredacted notification JSON through ordinary endpoints.

Housing and forum transactions

Both features are plain create, read, update and delete. Every call needs a verified ACTIVE user. Authors change only their own rows; admins may delete any row.

A. Housing listings

Action	Who	What the backend does
Post or edit	Author	Require listing_type, district_id, title and description; contact_details is optional. New listings start ACTIVE. After each save, queue the AI check (page 19).
Browse and view	Any user	ACTIVE listings only, filtered by district_id and listing_type, newest first, 20 per page. Return contact_details only if the poster filled it in. Authors also see their own CLOSED listings.
Close or reopen	Author	Switch ACTIVE and CLOSED. A closed listing leaves browsing but stays in the author's list.
Delete	Author or admin	Hard delete using section B.

B. Deleting a listing or a post

1. Check that the caller is the author or an ADMIN, then start one transaction. **2.** Close every open case on the content with the statement below (a question's answers are included). **3.** Delete the row; a question's answers go with it, and the foreign keys empty the case and notification links. **4.** If an admin deleted it, insert a MODERATION notification for the author with plain text and no link. Commit.

```
-- $1 post/listing id; $2 admin id and $3 note (both NULL when the author deletes)
UPDATE moderation_cases
SET status = 'SOLVED', resolution = 'CONTENT_DELETED',
    resolution_note = $3, resolved_at = now(), updated_at = now(),
    reviewed_by = coalesce(reviewed_by, $2)
WHERE status <> 'SOLVED'
  AND (target_post_id = $1 OR target_housing_id = $1
       OR target_post_id IN (SELECT id FROM forum_posts
                             WHERE parent_post_id = $1));
```

C. Forum questions and answers

Action	Who	What the backend does
Ask or edit a question	Author	Require category_id, title and body. Editing moves updated_at; kind and parent never change. Queue the AI check.
Answer	Any user	parent_post_id must be a QUESTION, never an answer; an answer has no title or category. In the same transaction insert a FORUM_REPLY notification for the question's author (skip it when answering your own question) with event_key reply:<answer id>. Send the email after commit; a failed email is logged and never undoes the answer. Queue the AI check.
List and open	Any user	Questions only, optional category_id filter, newest first, 20 per page. Opening a question returns it with its answers, oldest first.
Keyword search	Any user	Run the query on page 14 with the text as a bound parameter. At most 30 results; an answer hit links to its question.
Delete	Author or admin	Section B. Deleting a question deletes its answers.

Moderation and admin transactions

ADMIN is the only reviewer. Every admin call checks users.role = ADMIN. The auth middleware also reads users.status from the database on every request, so a banned (DEACTIVATED) user is refused at once, even with a valid JWT.

A. How cases are created

1. **User report.** A user names exactly one target (user, trip, listing or post) and a reason. Reject an empty reason, reporting yourself or your own content, and a second open report by the same user on the same target. Insert a USER case (ACTIVE). The reporter can read only ACTIVE, REVIEWING or SOLVED for their own reports.
2. **AI flag.** After a post or listing is saved, a background job sends its text to the AI provider, never inside the user's request or a database transaction. The job notes the content's updated_at first. If the text is judged unsuitable, read the row again: if it is gone or updated_at changed, drop the result; otherwise insert an AI case (ai_model, ai_score, reason) unless an open AI case on that target exists. A provider error or timeout is only logged; the content stays visible.
3. AI only creates cases. It never deletes content or bans users.

B. How an admin works a case

Step	Rule
See the queue	ACTIVE cases first, then REVIEWING, oldest first; filter by status or source. Show the reason, source, AI score and the reported content.
Take it	Run the statement below. If it updates 0 rows, another admin got there first: return 409.
Dismiss	Only the admin in reviewed_by. Set SOLVED, resolution DISMISSED, resolution_note and resolved_at.
Delete content	Only the reviewing admin, and only for a post or listing. Use the steps and statement on page 18, passing this admin and the note.
Ban the user	Only the reviewing admin. The user is the target, or the owner of the targeted content (author or trip driver). Run section C, then set the case SOLVED with USER_BANNED.

```
UPDATE moderation_cases
SET status = 'REVIEWING', reviewed_by = $2, updated_at = now()
WHERE id = $1 AND status = 'ACTIVE';
```

C. Ban, reactivate and change role (one transaction each)

Ban: set users.status = DEACTIVATED and insert an admin_audit_logs BAN row (reason, status before and after). You cannot ban yourself or another ADMIN; change the role first. **Reactivate:** set ACTIVE and insert a REACTIVATE row. **Change role:** switch USER and ADMIN and insert a CHANGE_ROLE row; you cannot change your own role, so at least one admin always remains.

A ban does not touch trips, bookings or payments. A DEACTIVATED driver cannot confirm rides, so an admin cancels their trips through the normal refund flow when needed.

D. Admin overview (read only)

Askıda bilet overview: counts and TL totals of sponsorship_requests per status, with paid totals taken from the ledger (payment_reconciliation). **Usage statistics:** counts of users, trips, bookings, listings and posts. Show totals only, never donor, recipient or trip identities. A test checks that the overview matches the ledger.

Prisma and implementation handoff

Create the initial migration

1. Represent the core tables in Prisma with explicit @map/@@map names. Declare every id as **String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid**; do not use autoincrement(). Use BigInt for monetary values and DateTime @db.Timestamptz for timestamps. Represent composite keys with @@id and compound uniqueness with @@unique. Map the delete rules with onDelete: Cascade (forum_posts parent) and onDelete: SetNull (notification and case post/listing references).
2. Generate a draft: **npx prisma migrate dev --create-only --name initial**. Review its generated SQL against pages 5-15; add missing CHECKs, views, functions, triggers and expression indexes. Do not append duplicate CREATE TABLE statements to Prisma-generated tables.
3. Apply with **npx prisma migrate dev**; commit schema.prisma, migration files and seed script together. Use **prisma migrate deploy** in deployment.
4. CHECKs, views and triggers need SQL migration coverage. Partial-index schema support depends on the pinned Prisma version; current documentation supports a where argument, while older projects may need raw migration SQL. Keep every rule in migration history. [S4]

Set @updatedAt on mutable models and update timestamps in raw SQL paths. BigInt cannot be serialized directly by JSON.stringify; send monetary minor units as decimal strings (or deliberately convert only safe integers). Use parameterized raw queries for row locking and keyword search. Never interpolate user input into SQL.

Essential integration checks before the demo

Check	Expected result
Email and access	Non-İTÜ sign-up rejected; expired code, reused code and a sixth wrong attempt rejected; unverified or DEACTIVATED user blocked; only an ADMIN can change roles or ban.
Trip integrity	Same-city or non-İstanbul route rejected; stale departure cannot be booked.
Last-seat race	Two concurrent attempts for one remaining seat produce one reservation, never overbooking.
Funding race and anonymity	Two donors claim one request: only one wins. Pool/receipt/driver-list/reply payloads never expose prohibited identities or trip data; a funded passenger never appears in the driver's list.
Payment reliability	Duplicate results create one PAYMENT/HOLD pair; timeout reconciles; cancellation + late success refunds without taking a seat.
Completion and ratings	Driver-only/passenger-only confirmation cannot release; both can. A single rating is absent from public lists/counts/averages, except a funded passenger's rating of the driver.
Ledger and audit	Reconciliation identity holds per payment. Ledger and audit edits fail.
Housing, forum and notifications	Only the author or an admin can edit or delete (others get 403). Answer-to-answer rejected; deleting a question deletes its answers; CLOSED listings are not browsable. A forum reply creates one in-app row and one email; a duplicate event creates no second notification.
Reports and AI flags	A duplicate open report is rejected; the reporter sees only the safe status. A stale AI flag is dropped. Deleting content that has an open case marks that case SOLVED.
Admin actions	A second admin taking a case gets 409. A DEACTIVATED user is refused even with a valid JWT. An admin cannot ban an admin or change their own role. Every ban, reactivation and role change writes an audit row.

Validation status: the core SQL (pages 5-15) and the two statements on pages 18-19 ran in page order on a scratch PostgreSQL 16 database, with constraint, cascade and view tests. The backend contracts are service code and were not executed; concurrency, provider and API checks must run in your environment. No live provider transaction or deployment was performed.

Traceability and references

Source requirements

[S1] Supplied “Software Engineering - Project Plan”, project İTÜ ShareMaxxing, submission date 25 October 2026, attached as Pasted markdown.md. Design basis: §1.1 Scope and assumptions; §1.2 D-01 to D-09; §1.3 T-01 to T-27; §1.4 Security, Reliability and Privacy. Optional items D-13, D-14, T-08, T-20, T-23 and T-34 are not designed here. The source plan is authoritative for project scope.

Technical references - checked 27 October 2026

[S2] PostgreSQL: Constraints
Row checks, uniqueness and foreign keys; cross-row rules need other mechanisms.
https://www.postgresql.org/docs/current/ddl-constraints.html

[S3] PostgreSQL: Explicit Locking
SELECT FOR UPDATE and consistent row-locking behavior.
https://www.postgresql.org/docs/current/explicit-locking.html

[S4] Prisma: Unsupported database features and custom migrations
Version-aware SQL migration support; preserve checks, views and triggers.
https://www.prisma.io/docs/orm/prisma-migrate/workflows/unsupported-database-features

[S5] iyzico: Marketplace item approval
Approval uses paymentTransactionId; confirm the selected sandbox adapter before implementing settlement.
https://docs.iyzico.com/en/getting-started/preliminaries/api-reference-beta/marketplace/approve

[S6] PostgreSQL: Full Text Search
Built-in token search and indexes for required forum keyword search.
https://www.postgresql.org/docs/current/textsearch.html

[S7] iyzico: Marketplace implementation
Sandbox payee mapping needs a submerchant key; each payment item supplies that key and its allocation.
https://docs.iyzico.com/en/products/marketplace/marketplace-implementation

Decisions to record before coding

Confirm the funded-seat policy (drivers see nothing, so no driver rating or no-show claim for funded seats) and public-history visibility; the per-passenger release rule; the 15-minute hold and funding-request expiry; and the cancellation/no-show resolution policy. These are proposals where the plan leaves behavior open. They do not block creating the shared tables, but they determine service logic and API fields. The source plan still lists a moderator role; this design has only USER and ADMIN, so update D-01, D-09, T-02 and T-25 in the plan to match.

Deliberately omitted

No chat messages, intermediate route stops, GPS tracks, driving-license records, real-money wallets, rent/deposit payments, financial-need documents, other-university accounts, a moderator role, moderation-action history, English-language content tables, a waitlist, forum votes, course-code tags and vector search. These are out of scope, optional in the source plan or removed from the design. No vehicle table is needed by the current trip requirements.

Build order: accounts and location seeds → trips/bookings → sandbox payments and funding → ratings/no-show → forum/housing → moderation/notifications/admin. Agree on these shared migrations before each developer builds their feature endpoints.